Syncner Learning Management System - Low-Level Design (LLD)

Document Information

- **Version**: 1.0
- **Date**: September 2025
- **Author**: Pranav Holla
- **Status**: Production Ready

Table of Contents

1. Component Design
2. Database Schema Design
3. API Design
4. Authentication Flow
5. Data Models
6. Service Layer Design
7. Frontend Architecture
8. State Management
9. Error Handling
10. Performance Optimization

Component Design

Frontend Component Architecture

1. Layout Components

```
// Layout.tsx - Main application layout
interface LayoutProps {
  children: ReactNode;
  sidebar?: boolean;
  header?: boolean;
}

// StudentLayout.tsx - Student-specific layout
interface StudentLayoutProps {
  children: ReactNode;
  currentPath?: string;
}

// StaffSidebar.tsx - Staff navigation sidebar
interface StaffSidebarProps {
  isOpen: boolean;
  onToggle: () => void;
}
```

2. Authentication Components

```
// ProtectedRoute.tsx - Route protection wrapper
interface ProtectedRouteProps {
  children: ReactNode;
  requireAdmin?: boolean;
  requireStaff?: boolean;
}

// LoginPage.tsx - User authentication
interface LoginFormData {
  email: string;
  password: string;
}

// RegisterPage.tsx - User registration
interface RegisterFormData {
  name: string;
  email: string;
  password: string;
  careerTrack: string;
}
```

3. Dashboard Components

```
// StudentDashboard.tsx - Student main dashboard
interface DashboardProps {
  user: User;
  levels: Level[];
  progress: ProgressData;
}

// AdminDashboard.tsx - Admin main dashboard
interface AdminDashboardProps {
  analytics: AnalyticsData;
  users: User[];
  systemStats: SystemStats;
}
```

4. Portfolio Components

```
// StudentPortfolio.tsx - Student portfolio display
interface StudentPortfolioProps {
  studentId?: string;
  studentData?: StudentPortfolioData;
```

```
}

// PublicPortfolio.tsx - Public portfolio access
interface PublicPortfolioProps {
  studentId: string;
}
```

Backend Component Architecture

1. Controller Layer

```
// authController.js - Authentication logic
class AuthController {
  async register(req, res) { /* Registration logic */ }
  async login(req, res) { /* Login logic */ }
  async logout(req, res) { /* Logout logic */ }
  async getProfile(req, res) { /* Profile retrieval */ }
  async updateProfile(req, res) { /* Profile update */ }
}

// portfolioController.js - Portfolio management
class PortfolioController {
  async getStudentPortfolio(req, res) { /* Portfolio generation */ }
  async getPublicPortfolio(req, res) { /* Public access */ }
}
```

2. Service Layer

```
// leaderboardService.js - Leaderboard calculations
class LeaderboardService {
  static async calculateStudentScore(userId, careerTrack) { /* Score
calculation */ }
  static async getLeaderboard(careerTrack, limit) { /* Leaderboard
generation */ }
  static async getStudentRank(userId, careerTrack) { /* Rank calculation
*/ }
}

// geminiService.js - AI integration
class GeminiService {
  async chatWithAI(message, context) { /* AI chat functionality */ }
}
```

3. Middleware Layer

```
// auth.js - Authentication middleware
const protect = async (req, res, next) => { /* Token validation */ }
const authorize = (...roles) => { /* Role-based access */ }

// errorHandler.js - Error handling middleware
const errorHandler = (err, req, res, next) => { /* Error processing */ }
```

Database Schema Design

User Collection

```
// User.js - User model
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: {
    type: String,
    enum: ['student', 'admin', 'staff'],
    default: 'student'
  },
  careerTrack: { type: String, required: true },
  college: { type: String },
  department: { type: String },
  phone: { type: String },
  location: { type: String },
  gpa: { type: String },
  graduationYear: { type: String },
  profileComplete: { type: Boolean, default: false },
  isActive: { type: Boolean, default: true },
  lastActive: { type: Date, default: Date.now },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
});
```

UserProgress Collection

```
// UserProgress.js - Progress tracking model
const userProgressSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  careerTrack: { type: String, required: true },
  currentLevel: { type: Number, default: 1 },
  completedLevels: [{
    levelId: { type: String, required: true },
    levelOrder: { type: Number, required: true },
    completedAt: { type: Date, default: Date.now },
    completedSections: {
```

```
      learn: { type: Boolean, default: false },
      test: { type: Boolean, default: false },
      exercise: { type: Boolean, default: false }
    }
  }],
  learnProgress: [{
    levelId: { type: String, required: true },
    timeSpent: { type: Number, default: 0 },
    linksAccessed: [String],
    isCompleted: { type: Boolean, default: false },
    completedAt: { type: Date }
  }],
  testProgress: [{
    levelId: { type: String, required: true },
    score: { type: Number, default: 0 },
    attempts: [{
      score: { type: Number, required: true },
      answers: [String],
      submittedAt: { type: Date, default: Date.now }
    }],
    isCompleted: { type: Boolean, default: false },
    completedAt: { type: Date }
  }],
  exerciseProgress: [{
    levelId: { type: String, required: true },
    submissions: [{
      links: [String],
      submittedAt: { type: Date, default: Date.now },
      status: {
        type: String,
        enum: ['submitted', 'approved', 'rejected'],
        default: 'submitted'
      },
      feedback: { type: String }
    }],
    isCompleted: { type: Boolean, default: false },
    completedAt: { type: Date }
  }]
});
```

Curriculum Collection

```
// Curriculum.js - Course content model
const curriculumSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  type: {
    type: String,
    enum: ['learn', 'test', 'exercise'],
    required: true
  },
```

```
  careerTrack: { type: String, required: true },
  level: { type: Number, required: true },
  order: { type: Number, required: true },
  content: { type: String, required: true },
  externalLink: { type: String },
  options: [String], // For test questions
  answer: { type: String }, // For test questions
  resources: [String], // For exercises
  isActive: { type: Boolean, default: true },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now }
});
```

Attendance Collection

```
// Attendance.js - Attendance tracking model
const attendanceSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
  date: { type: Date, required: true },
  status: {
    type: String,
    enum: ['present', 'absent'],
    required: true
  },
  markedAt: { type: Date, default: Date.now },
  markedBy: { type: String, default: 'system' }
});
```

API Design

Authentication Endpoints

```
// POST /api/auth/register
{
  "name": "string",
  "email": "string",
  "password": "string",
  "careerTrack": "string"
}

// POST /api/auth/login
{
  "email": "string",
  "password": "string"
}

// GET /api/auth/profile
// Headers: Authorization: Bearer <token>
```

```
// PUT /api/auth/profile
{
  "name": "string",
  "phone": "string",
  "location": "string",
  "college": "string",
  "department": "string"
}
```

Portfolio Endpoints

```
// GET /api/portfolio/me
// Headers: Authorization: Bearer <token>

// GET /api/portfolio/public/:studentId
// No authentication required

// GET /api/portfolio/student/:studentId
// Headers: Authorization: Bearer <token>
// Requires: staff or admin role
```

Curriculum Endpoints

```
// GET /api/curriculum/levels/:careerTrack
// Headers: Authorization: Bearer <token>

// GET /api/curriculum/level/:levelId
// Headers: Authorization: Bearer <token>

// POST /api/curriculum/level
// Headers: Authorization: Bearer <token>
// Requires: admin role
{
  "title": "string",
  "description": "string",
  "careerTrack": "string",
  "difficulty": "string",
  "skills": ["string"],
  "learn": [...],
  "test": [...],
  "exercise": [...]
}
```

Progress Endpoints

```
// GET /api/progress/levels
// Headers: Authorization: Bearer <token>

// POST /api/progress/learn
// Headers: Authorization: Bearer <token>
{
  "levelId": "string",
  "timeSpent": "number",
  "linksAccessed": ["string"]
}

// POST /api/progress/test
// Headers: Authorization: Bearer <token>
{
  "levelId": "string",
  "score": "number",
  "answers": ["string"]
}

// POST /api/progress/exercise
// Headers: Authorization: Bearer <token>
{
  "levelId": "string",
  "links": ["string"]
}
```

Authentication Flow

JWT Token Structure

```
// Token payload
{
  "userId": "string",
  "email": "string",
  "role": "string",
  "iat": "number",
  "exp": "number"
}

// Token generation
const generateToken = (id) => {
  return jwt.sign({ id }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRE || '30d'
  });
};
```

Authentication Middleware

```
const protect = async (req, res, next) => {
  let token;

  if (req.headers.authorization &&
      req.headers.authorization.startsWith('Bearer')) {
    token = req.headers.authorization.split(' ')[1];
  }

  if (!token) {
    return res.status(401).json({
      success: false,
      message: 'Not authorized to access this route'
    });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(decoded.id);
    next();
  } catch (error) {
    return res.status(401).json({
      success: false,
      message: 'Not authorized to access this route'
    });
  }
};
```

Role-Based Authorization

```
const authorize = (...roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({
        success: false,
        message: `User role ${req.user.role} is not authorized to access
this route`
      });
    }
    next();
  };
};
```

Data Models

Frontend Data Models

```
// User interface
interface User {
```

```
  id: string;
  _id?: string;
  email: string;
  name: string;
  role: 'student' | 'admin' | 'staff';
  college: string;
  department?: string;
  careerTrack?: string;
  profileComplete: boolean;
  level?: string;
  interests?: string[];
  streakDays?: number;
  totalPoints?: number;
  phone?: string;
  location?: string;
  gpa?: string;
  graduationYear?: string;
  lastActive?: string;
  isActive?: boolean;
}

// Level interface
interface Level {
  id: string;
  title: string;
  description: string;
  difficulty: string;
  duration: string;
  careerTrack: string;
  skills: string[];
  prerequisites: string[];
  learn: LearnItem[];
  test: TestItem[];
  exercise: ExerciseItem[];
  order: number;
  isActive: boolean;
  estimatedHours: number;
  progress?: number;
  status?: string;
  isUnlocked?: boolean;
  isCompleted?: boolean;
  completedSections?: {
    learn?: boolean;
    exercise?: boolean;
    test?: boolean;
  };
}

// Portfolio data interface
interface StudentPortfolioData {
  student: {
    _id: string;
    name: string;
    email: string;
```

```
    careerTrack: string;
    college?: string;
    department?: string;
    createdAt: string;
    introduction?: string | null;
    readyToHire: boolean;
    duration: string;
    skills?: string[];
    certificateId: string;
    recommendedRoles: string[];
  };
  progress: {
    currentLevel: number;
    completedLevels: number;
    totalLevels: number;
    percentage: number;
    totalPoints: number;
  };
  attendance: {
    totalDays: number;
    presentDays: number;
    percentage: number;
    currentStreak: number;
    longestStreak: number;
  };
  leaderboard: {
    score: number;
    rank: number | null;
    breakdown: {
      progressScore: number;
      attendanceScore: number;
      consistencyScore: number;
      timeScore: number;
    };
  };
  learningLevels: LearningLevel[];
}
```

Backend Data Models

```
// Progress calculation utilities
const calculateSectionProgress = (section, completedSections, submissions
= []) => {
  if (section.type === 'learn') {
    return completedSections.learn ? 100 : 0;
  } else if (section.type === 'test') {
    const latestSubmission = submissions[submissions.length - 1];
    return latestSubmission ? latestSubmission.score : 0;
  } else if (section.type === 'exercise') {
    const approvedSubmissions = submissions.filter(s => s.status ===
'approved');
```

```
    return approvedSubmissions.length > 0 ? 100 : 0;
  }
  return 0;
};

// Level progress calculation
const calculateLevelProgress = (level, completedSections, submissions) =>
{
  const sections = ['learn', 'test', 'exercise'];
  const totalProgress = sections.reduce((sum, sectionType) => {
    const section = level[sectionType];
    if (section && section.length > 0) {
      const sectionProgress = calculateSectionProgress(
        { type: sectionType },
        completedSections,
        submissions[sectionType] || []
      );
      return sum + sectionProgress;
    }
    return sum;
  }, 0);

  return Math.round(totalProgress / sections.length);
};
```

Service Layer Design

API Service Class

```
// api.ts - Centralized API service
class ApiService {
  private baseURL: string;

  constructor() {
    this.baseURL = import.meta.env.VITE_API_URL ||
'http://localhost:5000/api';
  }

  private getHeaders(token?: string): HeadersInit {
    const headers: HeadersInit = {
      'Content-Type': 'application/json',
    };

    if (token) {
      headers.Authorization = `Bearer ${token}`;
    }

    return headers;
  }

  private async handleResponse(response: Response) {
```

```
    if (!response.ok) {
      const error = await response.json();
      throw new Error(error.message || 'An error occurred');
    }
    return response.json();
  }

  // Authentication methods
  async login(credentials: { email: string; password: string }) {
    const response = await fetch(`${this.baseURL}/auth/login`, {
      method: 'POST',
      headers: this.getHeaders(),
      body: JSON.stringify(credentials),
    });
    return this.handleResponse(response);
  }

  // Portfolio methods
  async getStudentPortfolio(studentId: string = 'me'): Promise<any> {
    const token = localStorage.getItem('token');
    const response = await fetch(`${this.baseURL}/portfolio/${studentId}`,
{
      headers: this.getHeaders(token),
    });
    return this.handleResponse(response);
  }

  // Progress methods
  async updateLearnProgress(levelId: string, timeSpent: number,
linksAccessed: string[]) {
    const token = localStorage.getItem('token');
    const response = await fetch(`${this.baseURL}/progress/learn`, {
      method: 'POST',
      headers: this.getHeaders(token),
      body: JSON.stringify({ levelId, timeSpent, linksAccessed }),
    });
    return this.handleResponse(response);
  }
}
```

Portfolio Service

```
// portfolioService.ts - Portfolio-specific service
export class PortfolioService {
  static async getStudentPortfolio(studentId: string = 'me'):
Promise<StudentPortfolioData> {
    try {
      const response = await apiService.getStudentPortfolio(studentId);
      return response.data;
    } catch (error) {
      console.error('Error fetching portfolio:', error);
```

```
      throw error;
    }
  }

  static async getPublicPortfolio(studentId: string):
Promise<StudentPortfolioData> {
    try {
      const response = await
fetch(`${apiService.baseURL}/portfolio/public/${studentId}`);
      const data = await response.json();
      return data.data;
    } catch (error) {
      console.error('Error fetching public portfolio:', error);
      throw error;
    }
  }
}
```

Frontend Architecture

Component Structure

```
src/
├── components/
│   ├── Layout.tsx                # Main layout wrapper
│   ├── StudentLayout.tsx         # Student-specific layout
│   ├── ProtectedRoute.tsx        # Route protection
│   ├── StudentPortfolio.tsx      # Portfolio display
│   ├── EnhancedLearningTimeline.tsx # Learning timeline
│   └── admin/                    # Admin-specific components
├── pages/
│   ├── StudentDashboard.tsx      # Student dashboard
│   ├── AdminDashboard.tsx        # Admin dashboard
│   ├── LoginPage.tsx             # Authentication
│   └── staff/                    # Staff-specific pages
├── redux/
│   ├── store.ts                  # Redux store configuration
│   └── slices/                   # Redux slices
├── services/
│   ├── api.ts                    # API service
│   └── portfolioService.ts       # Portfolio service
└── utils/
    ├── routing.ts                # Routing utilities
    └── themeUtils.ts             # Theme utilities
```

Redux Store Structure

```
// store.ts - Redux store configuration
export const store = configureStore({
```

```
  reducer: {
    auth: authSlice.reducer,
    portfolio: portfolioSlice.reducer,
    curriculum: curriculumSlice.reducer,
    staff: staffSlice.reducer,
    analytics: analyticsSlice.reducer,
    settings: settingsSlice.reducer,
    theme: themeSlice.reducer,
    learningJourney: learningJourneySlice.reducer,
    gemini: geminiSlice.reducer,
    user: userSlice.reducer,
  },
  middleware: (getDefaultMiddleware) =>
    getDefaultMiddleware({
      serializableCheck: {
        ignoredActions: [FLUSH, REHYDRATE, PAUSE, PERSIST, PURGE,
REGISTER],
      },
    }),
});
```

State Management

Auth Slice

```
// authSlice.ts - Authentication state management
interface AuthState {
  user: User | null;
  token: string | null;
  isLoading: boolean;
  error: string | null;
  isAuthenticated: boolean;
  isRestoringSession: boolean;
}

const authSlice = createSlice({
  name: 'auth',
  initialState,
  reducers: {
    loginStart: (state) => {
      state.isLoading = true;
      state.error = null;
    },
    loginSuccess: (state, action) => {
      state.isLoading = false;
      state.user = action.payload.user;
      state.token = action.payload.token;
      state.isAuthenticated = true;
      state.error = null;
    },
    loginFailure: (state, action) => {
```

```
      state.isLoading = false;
      state.error = action.payload;
      state.isAuthenticated = false;
    },
    logout: (state) => {
      state.user = null;
      state.token = null;
      state.isAuthenticated = false;
      state.error = null;
    },
  },
});
```

Portfolio Slice

```
// portfolioSlice.ts - Portfolio state management
interface PortfolioState {
  data: StudentPortfolioData | null;
  loading: boolean;
  error: string | null;
  lastFetched: number | null;
}

const portfolioSlice = createSlice({
  name: 'portfolio',
  initialState,
  reducers: {
    fetchPortfolioStart: (state) => {
      state.loading = true;
      state.error = null;
    },
    fetchPortfolioSuccess: (state, action) => {
      state.loading = false;
      state.data = action.payload;
      state.lastFetched = Date.now();
      state.error = null;
    },
    fetchPortfolioFailure: (state, action) => {
      state.loading = false;
      state.error = action.payload;
    },
    updatePortfolioSection: (state, action) => {
      if (state.data) {
        const { levelId, sectionType, data } = action.payload;
        // Update specific section data
      }
    },
  },
});
```

Error Handling

Frontend Error Handling

```
// ErrorBoundary.tsx - React error boundary
class ErrorBoundary extends Component<Props, State> {
  public state: State = {
    hasError: false,
    error: null
  };

  public static getDerivedStateFromError(error: Error): State {
    return {
      hasError: true,
      error
    };
  }

  public componentDidCatch(error: Error, errorInfo: ErrorInfo) {
    console.error('Uncaught error:', error, errorInfo);
  }

  public render() {
    if (this.state.hasError) {
      return (
        <div className="min-h-[200px] p-6 flex items-center justify-
center">
          <div className="text-center">
            <h2 className="text-xl font-semibold mb-2">Component
Error</h2>
            <p className="mb-4">{this.state.error?.message || 'Something
went wrong'}</p>
            <button onClick={() => window.location.reload()}>
              Reload Page
            </button>
          </div>
        </div>
      );
    }

    return this.props.children;
  }
}
```

Backend Error Handling

```
// errorHandler.js - Express error handling middleware
const errorHandler = (err, req, res, next) => {
  let error = { ...err };
  error.message = err.message;
```

```
  // Log error
  console.error(err);

  // Mongoose bad ObjectId
  if (err.name === 'CastError') {
    const message = 'Resource not found';
    error = new ErrorResponse(message, 404);
  }

  // Mongoose duplicate key
  if (err.code === 11000) {
    const message = 'Duplicate field value entered';
    error = new ErrorResponse(message, 400);
  }

  // Mongoose validation error
  if (err.name === 'ValidationError') {
    const message = Object.values(err.errors).map(val => val.message);
    error = new ErrorResponse(message, 400);
  }

  res.status(error.statusCode || 500).json({
    success: false,
    error: error.message || 'Server Error'
  });
};
```

Performance Optimization

Frontend Performance

```
// Code splitting with React.lazy
const StudentDashboard = React.lazy(() =>
import('./pages/StudentDashboard'));
const AdminDashboard = React.lazy(() => import('./pages/AdminDashboard'));

// Memoization for expensive calculations
const MemoizedPortfolio = React.memo(StudentPortfolio);

// Virtual scrolling for large lists
import { FixedSizeList as List } from 'react-window';

// Image optimization
const OptimizedImage = ({ src, alt, ...props }) => (
  <img
    src={src}
    alt={alt}
    loading="lazy"
    decoding="async"
    {...props}
```

```
    />
  );
```

Backend Performance

```
// Database indexing
userSchema.index({ email: 1 });
userSchema.index({ careerTrack: 1, isActive: 1 });
userProgressSchema.index({ userId: 1, careerTrack: 1 });

// Query optimization
const getStudentsWithProgress = async (filters) => {
  return await User.aggregate([
    { $match: filters },
    {
      $lookup: {
        from: 'userprogresses',
        localField: '_id',
        foreignField: 'userId',
        as: 'progress'
      }
    },
    {
      $project: {
        name: 1,
        email: 1,
        careerTrack: 1,
        progress: { $arrayElemAt: ['$progress', 0] }
      }
    }
  ]);
};

// Caching strategy
const cache = new Map();
const getCachedData = (key, fetchFunction, ttl = 300000) => {
  const cached = cache.get(key);
  if (cached && Date.now() - cached.timestamp < ttl) {
    return cached.data;
  }

  const data = await fetchFunction();
  cache.set(key, { data, timestamp: Date.now() });
  return data;
};
```

Database Optimization

```
// Aggregation pipelines for complex queries
const getStudentAnalytics = async (studentId) => {
  return await UserProgress.aggregate([
    { $match: { userId: new mongoose.Types.ObjectId(studentId) } },
    {
      $lookup: {
        from: 'attendances',
        localField: 'userId',
        foreignField: 'userId',
        as: 'attendance'
      }
    },
    {
      $project: {
        currentLevel: 1,
        completedLevels: { $size: '$completedLevels' },
        totalLearnTime: { $sum: '$learnProgress.timeSpent' },
        averageTestScore: { $avg: '$testProgress.score' },
        attendancePercentage: {
          $multiply: [
            { $divide: [
              { $size: { $filter: { input: '$attendance', cond: { $eq:
['$$this.status', 'present'] } } } },
              { $size: '$attendance' }
            ] },
            100
          ]
        }
      }
    }
  ]);
};
```

Document Version: v1.0

Submitted On: September 2025